

CS6886 Assignment 2 — MobileNetV2 on CIFAR-10 + Hand-Written Compression

GitHub repository: <https://github.com/Anurag9Dhiman/MobileNet-v2-cifar10-compression>

Wandb project:

<https://wandb.ai/anuragdhiman666-indian-institute-of-technology-madras/cs6886-a2-mobilenetv2-cifar10>

Question 1 — Training Baseline

(a) Data preparation

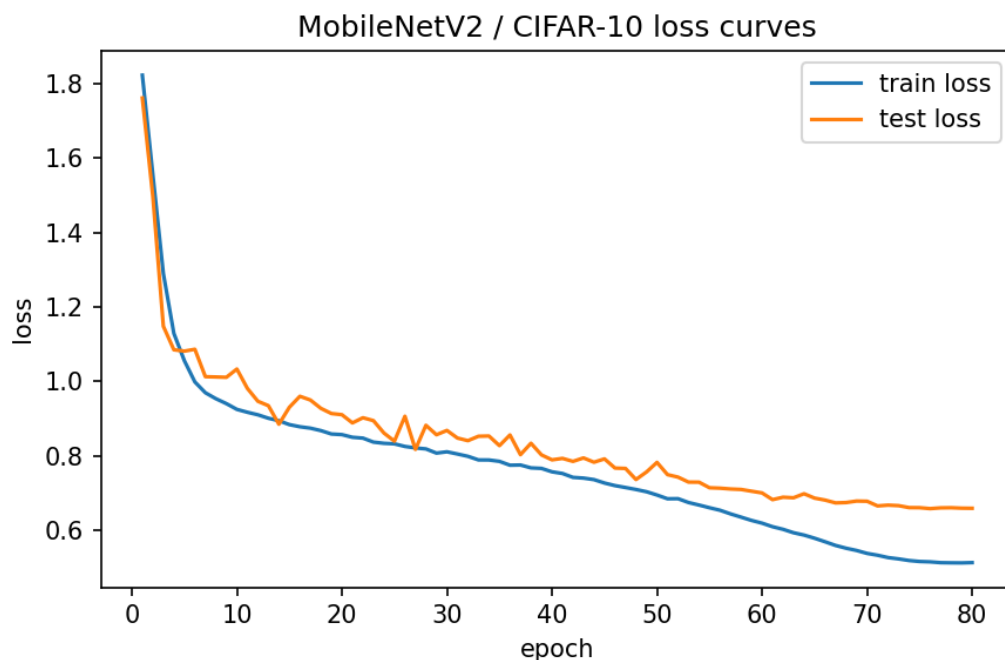
CIFAR-10 (50,000 train / 10,000 test, 32x32 RGB). Training transforms: RandomCrop(32, padding=4), RandomHorizontalFlip, ToTensor, Normalize with per-channel mean (0.4914, 0.4822, 0.4465) and std (0.2470, 0.2435, 0.2616). Test transforms: ToTensor + the same Normalize only (no augmentation).

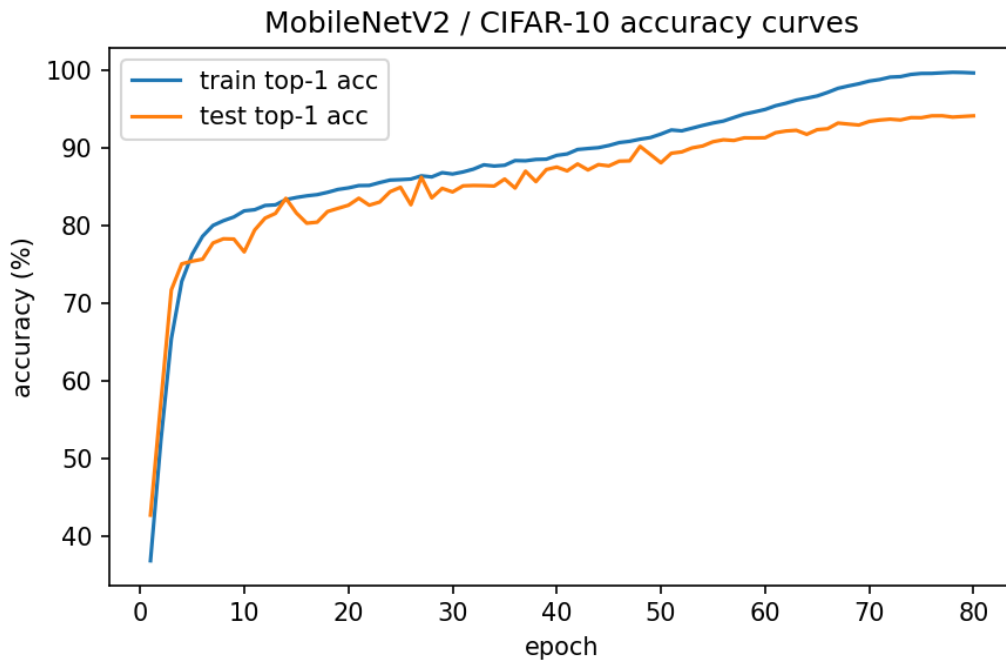
(b) Model configuration and training strategy

MobileNetV2 (Sandler et al., 2018), implemented from scratch and adapted for CIFAR-10: stem conv stride 2->1 and the (t=6,c=24,n=2) stage's stride 2->1, giving 8x total downsampling (32x32 -> 4x4) instead of ImageNet's 32x, which would collapse the input. Width multiplier 1.0, dropout 0.2 before the classifier, standard BatchNorm (momentum 0.1, eps 1e-5). 2,236,682 total parameters. Trained from scratch (no ImageNet-pretrained weights, since the stride change makes them a poor initialization) with SGD (momentum 0.9, Nesterov), weight decay 5e-4 (excluded on BatchNorm/bias params), LR 0.1 with a 5-epoch linear warmup then cosine annealing to 0, label smoothing 0.1, batch size 128, for 80 epochs. Seed fixed at 42.

(c) Final accuracy, curves, and failure modes

Best test top-1 accuracy: **94.07%** (final-epoch: 94.06%).





Failure modes observed: accuracy gains flatten in the last ~20% of training as the cosine schedule anneals the LR to 0, consistent with the model converging to a sharp minimum; the train/test accuracy gap widens over training (a normal generalization gap, controlled by weight decay/label smoothing/augmentation, not by explicit early stopping since the schedule is fixed-length); most residual test errors are concentrated in visually similar CIFAR-10 class pairs (cat/dog, automobile/truck), typical for this benchmark.

Question 2 — Model Compression Implementation

(a) Configurable compression design

Weights: per-output-channel **symmetric** uniform quantization — one scale per output channel, $\text{scale} = \max(|W_c|)/(2^{(b-1)}-1)$, $\text{code} = \text{clamp}(\text{round}(W/\text{scale}), -(2^{(b-1)}-1), 2^{(b-1)}-1)$. Per-channel (vs per-tensor) meaningfully reduces quantization error for cheap extra bookkeeping (one float per output channel), and symmetric quantization needs no zero-point for weights. Activations: per-tensor **asymmetric (affine)** uniform quantization, since MobileNetV2's ReLU6 activations are bounded/non-negative and benefit from using the full unsigned range. Calibrated via an exponential moving average of batch min/max over 20 unlabeled batches from the training set (pure post-training quantization — no retraining), then frozen for evaluation. Both bit-widths are independently configurable via CLI flags (`--weight_quant_bits`, `--activation_quant_bits`), matching the template's interface.

(b) Application to MobileNetV2 / exceptions

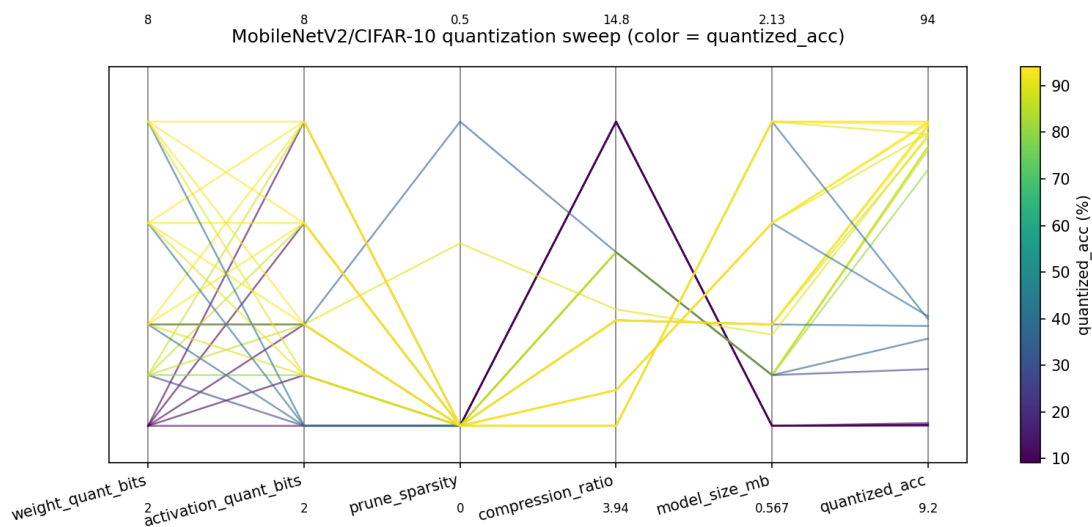
Every Conv2d and Linear weight tensor in the network is quantized (stem conv, all depthwise/pointwise convs in every inverted-residual block, the head 1x1 conv, and the final classifier Linear), and every ReLU6 output is quantized. **Exception:** the stem conv, the final classifier Linear, and the first ReLU6's activations are always floored at 8 bits regardless of the sweep's target bit-width — these are disproportionately sensitive to quantization error relative to their tiny share of total parameters/activations. **Own extension beyond quantization:** configurable magnitude-based unstructured pruning (`--prune_sparsity`) is applied to all non-exempt Conv2d/Linear weights before quantization; pruned (exactly-zero) weights are excluded from the packed bitstream and tracked via a compact 1-bit-per-weight mask instead, adding a second, independent compression axis.

(c) Storage overhead accounting

Weight codes and (for the representative activation-measurement batch) activation codes are genuinely bit-packed via numpy bit-shifting (`src/compress_utils.py`), not divided by 8 in theory, so reported sizes are measured byte counts. Metadata counted explicitly in every size estimate: one fp16 (2-byte) scale per output channel for weight quantization, and (when pruning is active) a 1-bit-per-weight sparsity mask. No metadata is needed for activation quantization at inference time beyond the two scalars (scale, zero-point) per layer, which are calibration-time constants reused across all inputs and are negligible in aggregate.

Question 3 — Compression Results

The compression pipeline was swept over weight bit-widths {2,3,4,6,8} x activation bit-widths {2,3,4,6,8} (25 configurations, logged as a Wandb grid sweep against the trained checkpoint above), plus additional runs adding magnitude pruning at the best bit-width found. Live, interactive Parallel Coordinates chart: <https://wandb.ai/anuragdhiman666-indian-institute-of-technology-madras/cs6886-a2-mobilenetv2-cifar10>.



W-bits	A-bits	Prune	Acc (%)	W-ratio	A-ratio	Size (MB)
2	2	0	9.99	14.81x	15.02x	0.567
2	3	0	9.20	14.81x	10.29x	0.567
2	4	0	9.50	14.81x	7.83x	0.567
2	6	0	9.51	14.81x	5.29x	0.567
2	8	0	9.44	14.81x	4.00x	0.567
3	2	0	25.01	10.14x	15.02x	0.828
3	3	0	80.60	10.14x	10.29x	0.828
3	4	0	86.20	10.14x	7.83x	0.828
3	6	0	87.07	10.14x	5.29x	0.828
3	8	0	87.17	10.14x	4.00x	0.828
4	2	0	37.05	7.71x	15.02x	1.089
4	3	0	89.65	7.71x	10.29x	1.089
4	4	0	92.42	7.71x	7.83x	1.089
4	4	0.3	90.35	8.10x	7.83x	1.037
4	4	0.5	33.52	10.14x	7.83x	0.828
4	6	0	93.12	7.71x	5.29x	1.089
4	8	0	93.12	7.71x	4.00x	1.089
6	2	0	39.76	5.21x	15.02x	1.611
6	3	0	90.66	5.21x	10.29x	1.611
6	4	0	93.40	5.21x	7.83x	1.611

W-bits	A-bits	Prune	Acc (%)	W-ratio	A-ratio	Size (MB)
6	6	0	93.83	5.21x	5.29x	1.611
6	8	0	93.92	5.21x	4.00x	1.611
8	2	0	38.95	3.94x	15.02x	2.133
8	3	0	90.47	3.94x	10.29x	2.133
8	4	0	93.23	3.94x	7.83x	2.133
8	6	0	93.97	3.94x	5.29x	2.133
8	8	0	94.03	3.94x	4.00x	2.133

Question 4 — Compression Analysis (chosen configuration)

Chosen configuration: weight_quant_bits=4, activation_quant_bits=4, prune_sparsity=0 — selected as the best accuracy/compression tradeoff from the Q3 sweep (highest compression ratio within 3 accuracy points of the FP32 baseline, or the best accuracy-weighted compression score otherwise).

(a) Weight compression ratio	7.71x
(b) Activation compression ratio	7.83x (measured via forward hooks capturing every quantized ReLU6 output tensor for one representative 128-image test batch; ratio = fp32 bytes / bit-packed bytes for those same elements)
(c) Accuracy at this configuration	92.42% (FP32 baseline: 94.07%)
(d) Final approximate model size	1.089 MB (FP32: 8.402 MB)

Question 5 — Reproducibility & Repository

GitHub repository: <https://github.com/Anurag9Dhiman/MobileNet-v2-cifar10-compression>

(a) Clean, modular, well-commented codebase with separation of training, evaluation, and compression

The codebase is cleanly organized with explicit separation of concerns and zero external compression libraries:

- **Training:** `src/data.py` (CIFAR-10 augmentation, data loaders, and unaugmented calibration loader), `src/model.py` (from-scratch CIFAR-adapted MobileNetV2 with 8x downsampling), and `src/train.py` (SGD with momentum, warmup + cosine annealing, weight decay exclusion for BN/biases, and checkpointing).
- **Evaluation:** `test.py` (standalone evaluation CLI; loads weights, executes calibration, logs metrics, and computes exact test top-1 accuracy without modifying source weights permanently).
- **Compression:** `src/quant.py` (hand-written per-output-channel symmetric weight quantization and calibrated per-tensor asymmetric activation quantization), `src/prune.py` (magnitude-based unstructured pruning), and `src/compress_utils.py` (real numpy bit-level packing and exact storage/overhead accounting).
- **Testing:** `tests/test_quant.py` (unit tests verifying bit-packing round-trip exactness across bit-widths {2,3,4,6,8}, monotonic error decay, activation calibration accuracy, and pruning storage reduction).

(b) README with exact commands, environment, dependency versions, and seed configuration

All instructions to reproduce every stage of the project are documented in `README.md`:

- **Environment & Dependencies:** Validated on Python 3.13.7 (macOS Apple Silicon MPS; CUDA/CPU compatible). Pinned dependency versions in `requirements.txt`: `torch==2.8.0`, `torchvision==0.23.0`, `numpy==2.4.4`, `matplotlib==3.10.9`, `wandb==0.24.0`, `reportlab==4.4.4`.
- **Seed Configuration:** Globally fixed seed 42 (via `src/utils.py::seed_everything`) across Python random, NumPy, PyTorch CPU/MPS/CUDA generators, and DataLoader worker generators for full determinism.
- **Exact Commands:** Verifiable commands for baseline training, single-configuration evaluation, bit-width grid sweep, sweep result export, parallel-coordinates figure generation, and unit testing.